

Auditoría de producción, Chaski sobre Solana + rieles WasiAI A2A

Solana LATAM Labs · registro consolidado de gaps (4 auditorías adversariales, 2026-07-21)

Objetivo: que no se escape nada. Código de producción escalable, no de hack. App de remesas de alto nivel sobre rieles de alto nivel 100% operativos.

0. Los 2 hallazgos estructurales (leer primero)

H1, El código actual es seguro *para EVM* porque se apoya en 3 primitivas que Solana NO da gratis:

1. El **nonce EIP-3009** como backstop *exactly-once* on-chain (revierte doble-gasto).
2. Direcciones **case-insensitive** (`toLowerCase()`) canonicaliza).
3. Firma **ECDSA** (`verifyMessage`).

Cada porteo "directo" de esas 3 asunciones **abre un vector**. Los 4 CRÍTICOS de seguridad son consecuencia de portear la asunción sin reemplazar la primitiva.

H2, Chaski hoy NO usa los rieles A2A. Llama a los agentes punto-a-punto (`chaski-v2/app/api/a2a/*`) → `fetch(REMIT_AGENTS_BASE_URL/.../invoke)`), con slugs hardcoded, sin discovery/orquestación/x402/fee-split del gateway. Si se apaga el gateway, Chaski sigue andando → señal de que no lo usa. **La promesa "app de remesas sobre los rieles de alto nivel" requiere trabajo nuevo, no solo hardening.**

1. Registro consolidado (priorizado, deduplicado de los 4 auditores)

CRÍTICO

#	Gap	Por qué rompe en producción	Cierre
CR-1	Drain del fee-payer gasless por partial-sign ciego	En Solana el fee-payer firma la tx ENTERA. Si firma un blob opaco, el atacante mete una instrucción que mueve el SOL/SPL del propio fee-payer → drena el relayer.	El fee-payer valida instrucción-por-instrucción antes de firmar (set exacto de ix, su pubkey nunca como source/authority, programId whitelist, compute acotado, blockhash fresco). → HU-SOL-14 (ampliada)
CR-2	<code>.toLowerCase()</code> sobre base58 → IDOR / ownership guard roto	base58 es case-sensitive: bajar a minúsculas corrompe la pubkey y colapsa owners distintos → cross-tenant leak (viola el Ownership Guard del CLAUDE.md). 15+ sitios (<code>supabase-settlement-ledger.ts:104,134,163</code> , <code>persistence.ts</code> , <code>authority.ts:83</code> , <code>kyc-store.ts</code> , <code>submit/route.ts:223</code>).	Canonicalización VM-aware : en <code>solana</code> NO lowercasear, comparar bytes vía <code>PublicKey</code> . → HU-SOL-7 (ampliada)
CR-3	Doble-release / doble-settle por fail-open de Redis	El fail-open era seguro <i>porque el nonce EIP-3009 revertía</i> . En Solana verify-only NO hay ese backstop → 2 llamadas a <code>/settle</code> / <code>release</code> con Redis caído → doble payout.	Exactly-once on-chain (state machine del escrow) + dedup durable (unique index Postgres sobre la signature) + fail-CLOSED para el adaptador Solana. → HU-SOL-6 + HU-SOL-12 (ampliadas)
CR-4	<code>release</code> a destino libre → una key robada drena TODO	Si la authority elige el <code>to</code> del release, comprometer <code>SOLANA_FEE_PAYER_KEY</code> /authority libera todos los escrows a la wallet del atacante (idéntico al bug #1 de WKH-191).	El beneficiary (depositAddress de TransFi) se graba en el deposit ; <code>release</code> solo transfiere a ese destino guardado. La authority elige <i>cuándo</i> , nunca <i>a quién</i> . → HU-SOL-12 (ampliada)
CR-5	Chaski bypassa los rieles A2A	La tesis "app sobre los rieles" no se cumple: sin discovery, sin x402, sin fee-split del gateway.	Enrutar quote+payout de Chaski por <code>POST /orchestrate</code> (o <code>/discover</code>) del gateway con agent-key. → HU-SOL-15 (nueva)
CR-6	<code>/invoke</code> de los agentes remit- sin auth	URLs públicas en Vercel; cualquiera invoca KYC/FX/payout gratis. El pago "lo enforces el gateway" que Chaski no usa (CR-5).	Guard de origen/shared-secret fail-closed en los <code>/invoke</code> + forzar el

#	Gap	Por qué rompe en producción	Cierre
	<i>ni x402 → billing evadible*</i>		pago por el gateway. → HU-SOL-16 (nueva)
CR-7	No hay refund trustless hoy	El <code>LedgerRefundGateway</code> devuelve un <code>refundTx</code> sintético y no revierte nada on-chain; una vez el USDC en TransFi, la recuperación es manual .	El escrow custodia el USDC y expone <code>refund</code> sender-callable tras deadline, funcionando aunque el facilitator esté caído. → HU-SOL-12 (ampliada)

ALTO

#	Gap	Cierre
AL-1	FX sin enforcement: el PEN entregado puede diferir del cotizado sin detección (la reconciliación entregado-vs-cotizado está muerta en el path real; TransFi off-rampea a su propia tasa, no se manda <code>destination.amount</code>).	HU-SOL-18 (nueva): pin de tasa/monto + activar <code>isDeliveredWithinReceiveTolerance</code> en el path real + alerta de drift.
AL-2	Doble libro de estado: el agregado client-side y el ledger server-side nunca se reconcilian con el webhook de TransFi (estado terminal real). El usuario ve "pendiente" para siempre.	HU-SOL-17 (nueva): estado on-chain autoritativo + backfill webhook → estado-usuario.
AL-3	Órfanos 'prepared' invisibles al reconcile + <code>reconcile-orphans</code> solo encola manual (no lee on-chain ni TransFi, no reintenta). Sin monitoreo de refunds pendientes.	HU-SOL-17 (nueva): reconcile que lee la verdad on-chain (vault) + estado TransFi, resuelve auto y alerta a #wasiai-alerts .
AL-4	Verificación ed25519 vs ECDSA (PoP/atestación): <code>verifyMessage</code> (viem) e <code>isAddress</code> no sirven en Solana; porteo ingenuo = DoS o check vacuo (firma forjada).	HU-SOL-8 (ampliada + OBLIGATORIA): verificador <code>nacl.sign.detached.verify</code> , decode base58 estricto, PoP mandatorio en el money-path (no opt-in).
AL-5	Wrong-mint / token falso / u64 truncado: anclar a símbolo/metadata en vez del mint pubkey → pagan con token basura; Token-2022 con transfer-fee → recibido ≠ enviado; <code>expectedValueMinor: number</code> trunca u64 >2 ⁵³ (footgun WKH-196).	HU-SOL-6 (ampliada): igualdad exacta de mint-pubkey + pin de program-id (SPL vs Token-2022) + monto por delta de balance de la ATA destino (no "exactly one transfer", que es EVM-específico) + bigint end-to-end.
AL-6	Blast-radius de la authority key: <code>OPERATOR_PRIVATE_KEY</code> single hot-key en env; si fee-payer = release-authority, un compromiso drena SOL (CR-1) + fuerza releases (CR-4).	HU-SOL-19 (nueva): keys separadas (fee-payer ≠ authority), multisig/timelock (Squads) fuera de devnet, KMS, rotación.
AL-7	Concurrencia del fee-payer sin diseño: N txs esponsorizadas comparten blockhash → colisión/expiry; el mutex EVM es single-instance.	HU-SOL-14 (ampliada): pool de durable-nonce accounts (o serialización + <code>lastValidBlockHeight</code>), rebroadcast ante expiry, commitment explícito.
AL-8	RPC Solana sin estrategia de producción (fallback/retries/commitment/expiry). El público de devnet no aguanta rate-limits.	HU-SOL-20 (nueva): RPC dedicado (Helius/Triton) + <code>_FALLBACK</code> + backoff + commitment por operación + probe en <code>/health</code> .
AL-9	Sin CI en chaski-v2 ni remit-agents (2 de 3 repos del money-path): el deploy Vercel es <code>next build</code> , no corre los <code>*.test.ts</code> .	HU-SOL-20 (nueva): CI bloqueante (typecheck+test+build) en ambos, copiando el <code>ci.yml</code> del facilitator.

#	Gap	Cierre
AL-10	Upgrade authority del programa Anchor indefinida = single point of catastrophic compromise (puede swapear el bytecode que custodia el USDC).	HU-SOL-12 + HU-SOL-19: mainnet → multisig/timelock o <code>set-upgrade-authority --final</code> post-auditoría. Gate.
AL-11	Rieles: sin timeout ni circuit-breaker en el invoke de agente (<code>compose.ts:874</code> sin AbortController; sin breaker por-agente). Un agente colgado pina capacidad y oscurece el settlement.	Hardening (HU-SOL-16): AbortController/timeout + breaker por-agente en compose/orchestrate.
AL-12	Fee-split e identidad EVM-only: un <code>payout_wallet</code> base58 falla <code>isValidWallet</code> → se re-rutea a plataforma (el creator Solana no cobra); ERC-8004 no bindea pubkey Solana (identidad/KYC del receptor Solana).	Parte del riel Solana → HU-SOL-19 / EPIC: payout no-EVM en el split + binding de identidad ed25519.
AL-13	Registro de agentes no reproducible (one-shot <code>POST /agents</code> a prod, sin script versionado) + agentes en mock (<code>PAYOUT_ALLOW MOCK</code> , wallets placeholder) + sin manifest self-served.	HU-SOL-16 (ampliada): script de registro idempotente + wallets reales testnet + salir de mock + <code>/.well-known/agent.json</code> .

MEDIO (hardening / diseño a especificar)

- **ME-1 · Pitfalls del programa Anchor** (todos deben cerrarse explícito en el SDD de HU-SOL-12): signer/authority + account confusion (`has_one` , seeds+bump, vault-authority=PDA); seeds `[b"escrow", depositor, remittance_id]` único; `init` (no `init_if_needed`) + `close` en terminal (anti re-init/revival); `overflow-checks = true` explícito en `Cargo.toml` + `checked_*` ; `token_program: Program<Token>` (anti CPI/program-id confusion); front-run del refund/release (transición terminal única, crear orden TransFi solo tras `release` finalized); rent del PDA/vault reclamado con `close` .
- **ME-2 · Idempotencia frágil ante re-quote:** `idempotencyKey = remittanceId:quoteId` ; un re-quote genera 2ª orden TransFi + 2º depositAddress sin cancelar la 1ª. → anclar idempotencia de la orden/escrow a `remittanceId` solo. (HU-SOL-17)
- **ME-3 · Replay cross-cluster:** las atestaciones atan `chainId:number` ; Solana no tiene chainId → el guard anti-replay ("400 por 0" con USDC de faucet) se re-abre. → bindear network-id CAIP-2 (`solana:devnet` vs `solana:mainnet`), nunca compartir el HMAC entre clusters. (HU-SOL-8)
- **ME-4 · Precisión latente:** `Number(r.value_minor)` reintroduce el bug WKH-196; `source.amount` a TransFi va como float mayor. → bigint/string end-to-end. (HU-SOL-17)
- **ME-5 · Rent-spam de escrows:** el **depositor** (no el fee-payer) paga el rent del escrow; rate-limit durable por identidad + presupuesto diario de SOL fail-closed; KYC/PoP antes de cualquier tx esponsorizada. (HU-SOL-14/19)

- **ME-6 · Observabilidad EVM-only:** el trípode gas/health/synthetic no cubre Solana; sin APM (Sentry) en ningún repo; sin métricas de escrow/gasless/RPC. → extender + APM. (HU-SOL-20)
- **ME-7 · Redis single-instance fail-open** en el money-path: Redis HA (Upstash multi-zona) como precondition de mainnet + revisar fail-open por-VM. (HU-SOL-20)
- **ME-8 · Config remit:** `TRANSFI_DEFAULT_NETWORK="base"` hardcode; `fx.ts` defaultea a URL **prod** mientras `payout.ts` a sandbox (inconsistencia peligrosa); sin webhook receiver `fund_settled` +HMAC (solo reporta `submitted`); sin `.env.example`. (HU-SOL-3 ampliada)
- **ME-9 · Cross-chain principal ↔ fee no modelado:** el leg principal (Solana) y el leg de fee (Avalanche) están desacoplados, sin agregado que los ate ni reconciliación. → objeto de remesa cross-rail. (HU-SOL-17)

2. Impacto en el backlog: 6 HUs nuevas + 5 ampliadas

Nuevas (cierran lo que se escapaba):

HU	Qué cierra
HU-SOL-15 · Chaski sobre los rieles A2A (<code>/orchestrate</code> / <code>/discover</code>)	CR-5
HU-SOL-16 · Auth + x402 en <code>/invoke</code> de agentes + timeout/breaker + registro reproducible	CR-6, AL-11, AL-13
HU-SOL-17 · Reconciliación real (on-chain autoritativo, webhook → estado, órfanos, cross-rail)	AL-2, AL-3, ME-2, ME-4, ME-9
HU-SOL-18 · FX enforcement + reconciliación entregado-vs-cotizado	AL-1
HU-SOL-19 · Key management + authority (keys separadas, multisig/timelock, KMS, upgrade authority)	AL-6, AL-10, AL-12
HU-SOL-20 · RPC producción + CI + observabilidad + Redis HA	AL-8, AL-9, ME-6, ME-7

Ampliadas (spec de seguridad mucho más profunda):

- **HU-SOL-12 (escrow Anchor):** beneficiary fijado en deposit, state machine exactly-once, refund deadline facilitator-independiente, todos los pitfalls Anchor (ME-1), close-rent, upgrade authority.
- **HU-SOL-14 (gasless):** validación instrucción-por-instrucción (CR-1), concurrencia durable-nonce (AL-7), funding/cap/rotación.
- **HU-SOL-7 (identidad):** canonicalización VM-aware base58, 15+ sitios `toLowerCase` (CR-2).
- **HU-SOL-8 (PoP):** ed25519 real + **OBLIGATORIO** en el money-path + network-id CAIP-2 (AL-4, ME-3).
- **HU-SOL-6 (verify):** mint-pubkey exacto + delta de balance ATA + bigint + fail-closed durable (AL-5, CR-3).

3. Lo que YA está sólido (production-grade, no tocar, replicar)

- **Rieles A2A:** auth money-path completa, **0 IDOR** (owner_ref en todo), SSRF excelente (DNS-rebinding guard, re-validación por-hop), x402 fee-settlement con re-verify on-chain, fee-split

money-safe (bps $\Sigma=10000$ fail-closed, refund-outbox, credit-back), observabilidad (`/health` , `/metrics`), Pino redactado, correlation id).

- **Facilitator EVM**: mutex de nonce, circuit breaker, RPC fallback por env, daily cap, preflight funding-low, health-probe cacheado. **Es el molde correcto**, el trabajo es portarlo a Solana reemplazando las 3 primitivas de H1.
- **Los 3 repos**: flags OFF por default + fail-loud, config env-driven fail-safe a testnet, higiene PII (enum-only), sin secrets hardcodeados, sin leak `NEXT_PUBLIC_` .

4. Realidad de scope (honesta)

El MVP "de alto nivel sobre rieles de alto nivel" que pedís es un **build serio**: la auditoría llevó el backlog de 14 → **~20 HUs** + profundizó 5. Es lo correcto para producción (no hack), pero **M5 (21-ago) no entra completo**. Recomendación de secuenciación:

1. **Núcleo money-path Solana correcto y seguro** (escrow con state machine + refund trustless, gasless con validación de ix, identidad base58 sin IDOR, verify fail-closed) → sin esto no hay MVP creíble.
2. **Chaski sobre los rieles** (HU-SOL-15/16) → cumple la tesis.
3. **Reconciliación + FX + ops** (HU-SOL-17/18/20) → producción.
4. **Key mgmt + auditoría externa** → gate antes de mainnet/plata real (después del 100% sandbox).

El Demo Day puede mostrar el núcleo (1+2) sólido en devnet; el resto es el camino a producción real, que igual va después del sandbox.

ADDENDUM, Segunda ola de auditoría (2026-07-21)

Dimensiones no cubiertas por la primera ola: compliance/PII, frontend/UX, datos/DB/RLS, testing/supply-chain

La primera ola cubrió money-path, seguridad, rieles, escalabilidad. Esta segunda ola audita las 4 superficies restantes. Confirma que la ingeniería técnica es fuerte; los gaps nuevos son de **capa regulatoria, UX de producción, infra de datos y prueba de la defensa**.

A. Compliance / PII / AML / regulatorio

CRÍTICO

- **CO-1** · `TRANSFI_USER_ID` **estático** (`payout.ts:96`): todas las remesas de todos los senders se atribuirían a UN solo usuario KYC de TransFi → patrón de **smurfing/estructuración**, rompe el AML del PSAV, viola el contrato del partner. **Decisión de arquitectura YA** (userId TransFi por-sender, creado al pasar el KYC Didit; cambia el shape de `PayoutInput`). → HU-SOL-22.

- **CO-2 · Travel Rule no se transmite** (`cashout-payout.ts:268-276` stub vacío; el body de TransFi ni incluye `travelRuleData`): el corredor queda no-conformante el día uno (Perú 2026). Falta capturar dirección/documento del originador y estructurar el payload. → HU-SOL-22.
- **CO-3 · Postura money-transmitter** (`chain.ts:81-87`): el flujo del principal *hoy* es custodial (wallet propia); el fix no-custodial (WKH-211/212) está flags-OFF. + **nota escrow**: nunca rutear el principal por un escrow que pueda leerse como custodia sin blindaje legal. → gate de go-live (flip) + memo legal (founder).

ALTO

- **CO-4 · Sin límites de monto ni monitoreo AML**; `riskLevel high` con KYC Approved **paga igual** (`decision.ts:60`); sin cola de revisión ni ROS; `aml.hits` fail-open latente. → HU-SOL-22.
- **CO-5 · Binding KYC ↔ pagador débil = teatro regulatorio**: un payout bajo el `verificationId` de otro haría que el reporte Travel Rule lleve datos de la persona equivocada (falsedad en registro). PoP **obligatorio fail-closed en prod.** → amplía HU-SOL-8 + HU-SOL-16.
- **CO-6 · PII inline al LLM de Anthropic** (`transform.ts:157` , `input-retry.ts:79`): DNI/beneficiario a un subprocesador no declarado; `a2a_events.goal / tasks` persisten PII. → marcar inputs `sensitive` (código) + DPA Anthropic (founder). → HU-SOL-23.
- **CO-7 · PII del beneficiario sin minimización ni retención** (localStorage completo sin TTL; huérfanas `ownerAddress:null` nunca se borran). → HU-SOL-23.

MEDIO / acción legal-founder (no código)

Consentimiento/política de privacidad inexistente + biométrico Didit (Ley 29733 Perú / BIPA EEUU); derecho de supresión incompleto (Didit/TransFi/Supabase); data residency + subprocesadores sin DPA + **registro del banco de datos ante la ANPD** (obligatorio Perú); Reg E (senders EEUU): decidir quién es el "remittance transfer provider"; matriz de monitoreo/ROS con TransFi-Didit; `legalId` crudo como `vendor_data` en Didit (`kyc.ts:43`) → referencia opaca. → **Track legal/founder** (memo + DPAs + registro).

B. Frontend / UX (app de alto nivel vs demo)

CRÍTICO (bloqueantes de "app de remesas", no polish)

- **UX-1 · Rechazo de firma = remesa muerta** (`confirm-and-send.ts:98-99,182`): persiste `confirmed` antes de firmar, el throw escapa crudo, y reintentar da `invalid_transition` ; la única salida borra el KYC. El gesto más común de un usuario cauto rompe todo.
- **UX-2 · Cerrar el browser = remesa invisible**: el estado se persiste pero ningún componente consume `ListHistory` (`grep listHistory =0`); sin tracking server-backed, la plata "desaparece" de la UI.
- **UX-3 · "Reembolsado" mentiroso**: `ledger-refund-gateway.ts` devuelve un refundTx sintético mostrado como real (`flow.tsx:739-741`). Con el escrow, falta la UI completa del refund trustless.

ALTO

KYC fallido → dead-end (retry imposible, FSM terminal); vista de fallo sin CTA; tracking sin timeout/ETA/link a explorer/stall-detection; sin manejo de cambio de cuenta/desconexión + `wrong_chain` sin copy; sin countdown de expiración del quote. Para el wallet-adapter Solana (a construir): ninguno de estos casos existe como patrón reusable.

MEDIO

Estado solo-en-browser (sin cross-device); a11y incompleta (banner de error sin `aria-live`, `maximumScale:1` bloquea pinch-zoom WCAG, contraste $\sim 3.4:1 < 4.5$); **i18n: voseo rioplatense para usuarios peruanos** (que tutean); recibo demo-level (sin fecha/tx/explorer/compartir, el sender le manda el comprobante al beneficiario por WhatsApp); PWA a medias (`icons:[]` , sin service worker); sin links a T&C/privacidad/soporte. → **HU-SOL-21** (UX producción + i18n + a11y + PWA).

C. Datos / DB / RLS / migraciones / backup

CRÍTICO

- **DB-1 · Sin tracking de migraciones aplicadas** → drift repo ↔ prod estructural (incidente documentado: WKH-155/164 aplicado solo en bdwv, caldz abierto meses). Humano como motor de migraciones; el applier está congelado en abril (13 de ~50). → HU-SOL-26.
- **DB-2 · RLS real no existe en ninguna tabla del money-path** (solo deny-all para anon; el 100% corre con service key BYPASSRLS; el IDOR base58 pasó *a través* de esta RLS). 5 tablas huérfanas sin ni ENABLE (`a2a_protocol_fees` , `a2a_events` , nonces, `facilitator_*`). Es WKH-SEC-02 pendiente. → HU-SOL-26.
- **DB-3 · Backup/DR contradictorio, restore drill diferido (WKH-76), 3 DBs de dinero sin postura:** perder Supabase entre backups = no saber a quién se le debe un payout = pérdida financiera directa. → HU-SOL-26.

ALTO

Retención inexistente (audit_log PII 90d no implementado; nonces/eventos infinitos); fee-ledger sin FK ni check `Σlegs ≤ total` ; `remittance_settlements` con read-modify-write no atómico en `attempts` , webhook que descarta `'prepared'` , unique `tx_hash` global no `(chain_id,tx_hash)` ; guard multi-tenant por convención con `getKeyById/getParentKey` haciendo `select('*')` solo por id. → HU-SOL-26 + amplía HU-SOL-17/HU-SOL-7.

D. Testing / supply-chain / contratos cross-repo

CRÍTICO

- **T-1 · El programa Anchor no tiene diseño de testing**, la pieza que custodia el dinero. Los CRÍTICOS del escrow solo se prueban contra la VM real. Falta `anchor test` + **LiteSVM** (manipular clock para el deadline del refund) + cada pitfall = test adversarial nombrado + `proptest` + coverage gate. → amplía HU-SOL-12.
- **T-2 · El gasless no se puede testear con mocks:** el bypass vive en cómo web3.js compila el mensaje (versioned tx, lookup tables). Hay que tirarle **transacciones maliciosas reales**. → amplía HU-SOL-14.

- **T-3 · Cero test de integración cross-repo**; los contratos son validadores manuales duplicados que driftean en silencio; dinero como `number` float otra vez. → HU-SOL-24 (contratos tipados + IDL versionado + golden EVM tests).
- **T-4 · chaski y remit sin CI, sin coverage, sin git remote** (código solo en esta máquina, sin backup). → HU-SOL-20 elevado: remotes + CI hoy.

ALTO

Vulns conocidas en deps de prod (`ws` / `axios` high en chaski, `next` high en remit); **precedente dic-2024** `@solana/web3.js` con **malware roba-llaves**, la dep exacta, en el repo con la fee-payer key → **pinning exacto + ignore-scripts** ; sin **verified build** del programa (gate de auditoría inauditable); e2e existente no reproducible/seguro → **harness devnet temprano (Sprint 1-2)**, nightly en CI; "EVM byte-idéntico" garantizado por review manual → **golden/snapshot tests**. → HU-SOL-25 (supply-chain) + amplía HU-SOL-12/20.

Nuevas HUs de la segunda ola

HU	Cierra
HU-SOL-21 · UX de producción (error states, recovery, tracking server-backed, i18n, a11y, PWA)	UX-1..3 + ALTO/MEDIO UX
HU-SOL-22 · Compliance data (TransFi userId por-sender, Travel Rule, límites, AML gating)	CO-1, CO-2, CO-4
HU-SOL-23 · PII/consent/retención (minimización, consent UI, DSR, mark-sensitive anti-LLM)	CO-6, CO-7, MEDIOS PII
HU-SOL-24 · Contratos A2A tipados + IDL versionado + golden EVM tests	T-3, T-9
HU-SOL-25 · Supply-chain hardening (pinning, ignore-scripts, overrides, npm/cargo audit, verified build)	T-5, T-6, T-7
HU-SOL-26 · DB de producción (migration tracking, RLS real WKH-SEC-02, backup/DR, retención)	DB-1, DB-2, DB-3, A1/A2/A3/A4 datos
Track legal/founder (no código)	memo MT/Reg E + escrow custody, DPAs, ANPD, matriz ROS

Ampliadas por la 2ª ola: HU-SOL-12 (testing Anchor + verified build), HU-SOL-14 (tests adversariales gasless), HU-SOL-20 (git remotes + CI hoy + harness devnet temprano), HU-SOL-8/16 (PoP obligatorio fail-closed), HU-SOL-17 (state-machine del ledger).

Backlog total: ~26 HUs + track legal. Nada se difiere; dinero no real hasta producción con auditoría.

Decisión de scope (founder, 2026-07-21)

- **El app se construye listo para producción:** las **27 HUs de código** (HU-SOL-1..27) se resuelven a calidad producción, incluidas todas las de las 2 auditorías + **HU-SOL-27** (load/chaos

testing + DR runbooks + gate de production-readiness).

- **Único diferido: el track legal (WKH-230)**, no se toca legal por ahora. Pero los **hooks de código** que legal habilita (Travel Rule en HU-SOL-22, PII/consent en HU-SOL-23, límites/AML) SÍ se construyen production-ready, listos para activarse.
- **TransFi como único off-ramp** (sin fallback/Conduit), simplifica sin bajar calidad.
- **Dinero no real** (devnet + sandbox) hasta el flip a plata real, que requiere legal + auditoría externa del programa Anchor.

Total: 27 HUs de código (production-ready) + 1 track legal diferido.